

JavaScript Error Handling Basics

Section 12 — Error Handling

JavaScript program চলার সময় কোনো unexpected problem হলে সেটাকে properly detect, handle এবং communicate করার জন্য Error Handling ব্যবহার করা হয়।

1. Error কী?

Error হলো এমন একটি সমস্যা যার কারণে JavaScript code expectedভাবে কাজ করতে পারে না।

Example:

```
console.log(username);
```

যদি `username` declare করা না থাকে, JavaScript error দেবে:

```
ReferenceError
```

2. Error vs Bug

দুটো এক জিনিস নয়।

Bug

Programmer-এর code-এর ভুল।

```
const price = 100;  
const quantity = 2;  
  
const total = price + quantity;
```

এখানে code technically valid, কিন্তু যদি উদ্দেশ্য হয় multiplication করা, তাহলে এটা একটি **logic bug**।

Error

JavaScript কোনো invalid operation detect করলে error তৈরি করতে পারে।

```
const user = null;

console.log(user.name);
```

এখানে `TypeError` হতে পারে।

সহজভাবে

```
Bug
↓
Code-এর ভুল

Error
↓
JavaScript/runtime সেই সমস্যা কে report করে
```

3. Error-এর প্রধান দুই ধরনের ধারণা

JavaScript শেখার সময় প্রথমে এই দুইটা distinction মনে রাখো:

Syntax Error

Code-এর syntax invalid।

```
const = 10;
```

JavaScript code parse করতে পারবে না।

Runtime Error

Code validভাবে parse হয়েছে, কিন্তু execution-এর সময় সমস্যা হয়েছে।

```
const user = null;

console.log(user.name);
```

এখানে execution-এর সময় `TypeError` হতে পারে।

4. try এবং catch

Error handle করার সবচেয়ে basic mechanism হলো:

```
try {  
  // risky code  
} catch (error) {  
  // error handling  
}
```

Example:

```
try {  
  const result = JSON.parse("invalid json");  
} catch (error) {  
  console.log("Something went wrong");  
}
```

এখানে `JSON.parse()` error করলে program পুরোপুরি crash না করে `catch` block-এ চলে যাবে।

5. try Block

`try` block-এর ভিতরে এমন code রাখা হয় যেটাতে error হওয়ার possibility আছে।

```
try {  
  const data = JSON.parse(input);  
}
```

যদি error হয়:

```
try  
↓  
error  
↓  
catch
```

6. catch Block

`catch` block error handle করার জন্য।

```
try {  
  JSON.parse("invalid");  
} catch (error) {  
  console.log("Invalid JSON");  
}
```

এখানে `error` হলো caught error object।

7. Error Object

`catch` থেকে পাওয়া error object-এর গুরুত্বপূর্ণ properties:

```
try {  
  JSON.parse("hello");  
} catch (error) {  
  console.log(error.name);  
  console.log(error.message);  
  console.log(error.stack);  
}
```

সাধারণভাবে:

```
error.name  
error.message  
error.stack
```

8. error.name

Error-এর type/name জানতে:

```
try {  
  JSON.parse("invalid");  
} catch (error) {
```

```
    console.log(error.name);  
  }
```

Output হতে পারে:

```
SyntaxError
```

9. error.message

Error-এর human-readable message:

```
try {  
  JSON.parse("invalid");  
} catch (error) {  
  console.log(error.message);  
}
```

এখানে JavaScript error-এর specific message পাওয়া যাবে।

10. error.stack

Debugging-এর সময় `stack` অনেক useful।

```
try {  
  JSON.parse("invalid");  
} catch (error) {  
  console.log(error.stack);  
}
```

Stack trace সাধারণত দেখায়:

- error কোথায় হয়েছে
- কোন function call-এর মাধ্যমে সেখানে পৌঁছেছে
- call stack-এর information

Production UI-তে পুরো stack user-কে দেখানো উচিত নয়।

11. try / catch Execution Flow

```
try {  
  console.log("A");  
  
  throw new Error("Something went wrong");  
  
  console.log("B");  
} catch (error) {  
  console.log("C");  
}
```

Output:

```
A  
C
```

কারণ `throw` হওয়ার পর `try` block-এর পরের code আর execute হয় না।

12. finally

`finally` block সাধারণত cleanup বা final action-এর জন্য ব্যবহার করা হয়।

Syntax:

```
try {  
  // risky code  
} catch (error) {  
  // handle error  
} finally {  
  // cleanup  
}
```

Example:

```
try {  
  console.log("Operation started");  
} catch (error) {  
  console.log("Error");  
} finally {
```

```
    console.log("Operation finished");  
}
```

Output:

```
Operation started  
Operation finished
```

13. Error হলেও `finally` Execute হয়

```
try {  
    throw new Error("Failed");  
} catch (error) {  
    console.log("Error handled");  
} finally {  
    console.log("Cleanup");  
}
```

Output:

```
Error handled  
Cleanup
```

14. Error না হলেও `finally` Execute হয়

```
try {  
    console.log("Success");  
} catch (error) {  
    console.log("Error");  
} finally {  
    console.log("Cleanup");  
}
```

Output:

Success
Cleanup

তাই `finally` -কে ভাবতে পারো:

শেষে যেটা করতেই হবে।

15. `finally` Real-World Use Case

ধরো API request-এর সময় loading state:

```
Start request
  ↓
Loading = true
  ↓
API request
  ↓
Success / Error
  ↓
Loading = false
```

Conceptually:

```
try {
  await fetchData();
} catch (error) {
  console.log(error);
} finally {
  setLoading(false);
}
```

Error হোক বা success হোক, loading বন্ধ করতে হবে।

16. `throw` কী?

নিজে থেকে একটি error তৈরি করে execution থামাতে `throw` ব্যবহার করা যায়।

```
throw new Error("Something went wrong");
```

এটা error তৈরি করে throw করে।

17. throw new Error()

Recommended pattern:

```
throw new Error("Invalid user");
```

তারপর সেটা catch করা যায়:

```
try {  
  throw new Error("Invalid user");  
} catch (error) {  
  console.log(error.message);  
}
```

Output:

```
Invalid user
```

18. throw কেন ব্যবহার করব?

যখন function বুঝতে পারে যে কোনো operation valid নয়, তখন caller-কে জানাতে throw করা যায়।

Example:

```
function withdraw(balance, amount) {  
  if (amount > balance) {  
    throw new Error("Insufficient balance");  
  }  
  
  return balance - amount;  
}
```

19. throw + try/catch

```
function withdraw(balance, amount) {  
  if (amount > balance) {  
    throw new Error("Insufficient balance");  
  }  
  
  return balance - amount;  
}  
  
try {  
  const result = withdraw(1000, 1500);  
  
  console.log(result);  
} catch (error) {  
  console.log(error.message);  
}
```

Output:

Insufficient balance

20. throw vs return

এটা খুব important।

return

Normal result পাঠায়:

```
function divide(a, b) {  
  return a / b;  
}
```

throw

Abnormal/error condition signal করে:

```
function divide(a, b) {
  if (b === 0) {
    throw new Error("Cannot divide by zero");
  }

  return a / b;
}
```

Mental model:

```
return
↓
Normal result

throw
↓
Error condition
```

21. Custom Validation Error

Real application-এ validation-এর জন্য `throw` খুব useful।

```
function registerUser(username, age) {
  if (!username) {
    throw new Error("Username is required");
  }

  if (age < 18) {
    throw new Error("User must be 18 or older");
  }

  return "Registration successful";
}
```

22. Error Propagation

Error সবসময় যেখানে তৈরি হয়েছে সেখানেই catch করতে হবে—এমন নয়।

Error caller-এর দিকে propagate করতে পারে।

Example:

```
function inner() {  
  throw new Error("Something went wrong");  
}  
  
function outer() {  
  inner();  
}  
  
try {  
  outer();  
} catch (error) {  
  console.log(error.message);  
}
```

Flow:

```
inner()  
  ↓  
throw Error  
  ↓  
outer()  
  ↓  
caller  
  ↓  
catch
```

23. Error Propagation Mental Model

```
Function A  
  ↓  
Function B  
  ↓  
Function C  
  ↓  
Error  
  ↑  
  |  
propagates upward  
  |
```

↑
try/catch

যে level-এ appropriate error handling করা সম্ভব, সেখানে catch করা যায়।

24. try/catch কোথায় রাখা উচিত?

সব code:

```
try {  
  // entire application  
} catch (error) {  
}
```

এভাবে wrap করা ভালো practice নয়।

বরং যেই operation-এ error expected বা meaningfulভাবে handle করা দরকার, সেখানে ব্যবহার করো।

25. JSON.parse() Error Handling

এটা খুব common example।

```
const input = '{"name":"Shariar"}';  
  
try {  
  const user = JSON.parse(input);  
  
  console.log(user.name);  
} catch (error) {  
  console.log("Invalid JSON");  
}
```

26. Invalid JSON

```
const input = "{name: Shariar}";  
  
try {
```

```
const user = JSON.parse(input);

console.log(user);
} catch (error) {
  console.log("Could not parse JSON");
}
```

এখানে `JSON.parse()` `SyntaxError` throw করতে পারে।

27. Error Types

JavaScript-এর কিছু important built-in error:

```
Error
├─ TypeError
├─ ReferenceError
├─ SyntaxError
├─ RangeError
├─ URIError
├─ EvalError
└─ AggregateError
```

সবচেয়ে বেশি practical:

- `TypeError`
- `ReferenceError`
- `SyntaxError`
- `RangeError`

28. TypeError

যখন কোনো value-এর type অনুযায়ী operation valid নয়, তখন `TypeError` হতে পারে।

Example:

```
const user = null;

console.log(user.name);
```

এখানে `null` থেকে property access করার চেষ্টা করা হচ্ছে।

Another Example

```
const number = 10;

number();
```

একটি number-কে function হিসেবে call করার চেষ্টা করা হচ্ছে।

এখানে `TypeError` হতে পারে।

29. ReferenceError

যখন JavaScript এমন identifier access করতে যায় যার binding পাওয়া যাচ্ছে না, তখন `ReferenceError` হতে পারে।

```
console.log(username);
```

যদি `username` কোথাও declare করা না থাকে:

```
ReferenceError
```

TDZ-ও `ReferenceError` দিতে পারে

```
console.log(age);

let age = 31;
```

এখানেও:

```
ReferenceError
```

কারণ `age` TDZ-তে আছে।

30. SyntaxError

JavaScript syntax invalid হলে `SyntaxError` হয়।

Example:

```
const = 10;
```

আরেকটি example:

```
if (true {  
  console.log("Hello");  
}
```

Parenthesis/bracket structure invalid।

31. SyntaxError কখন ঘটে?

অনেক `SyntaxError` code execute হওয়ার আগেই parser detect করে।

যেমন:

```
const name = ;
```

JavaScript এই code parse করতে পারবে না।

32. RangeError

যখন value-এর type ঠিক আছে কিন্তু value/range allowed সীমার বাইরে চলে যায়, তখন `RangeError` হতে পারে।

Example:

```
const arr = new Array(-1);
```

Negative array length valid নয়।

তাই `RangeError` হতে পারে।

Another Example

```
(10).toFixed(200);
```

`toFixed()` -এর digits-এর allowed range-এর বাইরে গেলে `RangeError` হতে পারে।

33. Common Error Types Comparison

Error	Meaning	Example
<code>TypeError</code>	Wrong type-এর operation	<code>null.name</code>
<code>ReferenceError</code>	Identifier পাওয়া যায়নি	<code>console.log(x)</code>
<code>SyntaxError</code>	Invalid syntax	<code>const = 10</code>
<code>RangeError</code>	Value valid range-এর বাইরে	<code>new Array(-1)</code>

34. `Error` Base Object

সব common errors-এর base হিসেবে `Error` object conceptually গুরুত্বপূর্ণ।

```
const error = new Error("Something went wrong");  
  
console.log(error.name);  
console.log(error.message);
```

Output:

```
Error  
Something went wrong
```

35. `new Error()` বনাম `Specific Error`

Generic:

```
throw new Error("Invalid input");
```

Specific:

```
throw new TypeError("Expected a number");
```

Specific error ব্যবহার করলে error-এর nature আরও clearly communicate করা যায়।

36. `TypeError` Manually Throw করা

```
function add(a, b) {  
  if (typeof a !== "number") {  
    throw new TypeError("a must be a number");  
  }  
  
  if (typeof b !== "number") {  
    throw new TypeError("b must be a number");  
  }  
  
  return a + b;  
}
```

37. Error Type Check করা

```
try {  
  throw new TypeError("Invalid type");  
} catch (error) {  
  if (error instanceof TypeError) {  
    console.log("Type error occurred");  
  }  
}
```

`instanceof` দিয়ে error type check করা যায়।

38. Multiple Error Types Handle করা

```
try {  
  // risky operation  
} catch (error) {  
  if (error instanceof TypeError) {  
    console.log("Type problem");  
  } else if (error instanceof ReferenceError) {  
    console.log("Reference problem");  
  } else {  
    console.log("Unknown error");  
  }  
}
```

39. Other Built-in Errors

URIError

URI-related operation invalid হলে হতে পারে।

```
decodeURIComponent("%");
```

এ ধরনের invalid URI encoding-এর ক্ষেত্রে **URIError** হতে পারে।

AggregateError

একাধিক error একসাথে represent করার জন্য।

বিশেষ করে Promise-related operations-এর ক্ষেত্রে useful।

Conceptually:

```
new AggregateError([  
  new Error("First error"),  
  new Error("Second error")  
]);
```

EvalError

`eval()` -related historical error type ।

Modern JavaScript development-এ সাধারণত এটি practically important নয় ।

40. `catch` Without Error Variable

Modern JavaScript-এ চাইলে error variable না নিয়েও `catch` করা যায় ।

```
try {
  JSON.parse("invalid");
} catch {
  console.log("Invalid JSON");
}
```

যখন error details দরকার নেই, এটা clean হতে পারে ।

41. Nested `try/catch`

প্রয়োজনে nested error handling করা যায় ।

```
try {
  try {
    JSON.parse("invalid");
  } catch (error) {
    console.log("Inner error");
  }
} catch (error) {
  console.log("Outer error");
}
```

Inner `catch` error handle করে ফেললে outer `catch` সাধারণত সেটা আর পাবে না ।

42. Re-throwing an Error

কখনো error locally log করে আবার উপরের layer-এ পাঠাতে চাইতে পারো ।

```

try {
  riskyOperation();
} catch (error) {
  console.log("Logging error");

  throw error;
}

```

এটাকে **re-throwing** বলা যায়।

Flow:

```

Error
↓
catch
↓
log / add context
↓
throw again
↓
upper layer

```

43. API Error Handling — Basic Pattern

```

async function getUser() {
  try {
    const response = await fetch("/api/user");

    if (!response.ok) {
      throw new Error("Failed to fetch user");
    }

    const data = await response.json();

    return data;
  } catch (error) {
    console.error(error);
  }
}

```

এখানে গুরুত্বপূর্ণ বিষয়:

`fetch()` HTTP 404 বা 500 response পেলেই নিজে থেকে সবসময় reject করে না।

তাই:

```
if (!response.ok) {  
  throw new Error("Request failed");  
}
```

নিজে error signal করা useful।

44. React Context — Loading/Error Pattern

React application-এ সাধারণ pattern:

```
Loading  
↓  
API request  
↓  
Success → Show data  
|  
└ Error → Show error
```

State concept:

```
const [loading, setLoading] = useState(true);  
const [error, setError] = useState(null);  
const [data, setData] = useState(null);
```

Request:

```
try {  
  setLoading(true);  
  
  const response = await fetch("/api/users");  
  
  if (!response.ok) {  
    throw new Error("Failed to load users");  
  }  
  
  const result = await response.json();
```

```
    setData(result);  
  } catch (error) {  
    setError(error.message);  
  } finally {  
    setLoading(false);  
  }
```

এটা frontend development-এ খুব common pattern ।

45. Empty `catch` কেন **Avoid** করা উচিত?

❌ Bad:

```
try {  
  riskyOperation();  
} catch {}
```

এখানে error silently disappear হয়ে যাচ্ছে ।

Better:

```
try {  
  riskyOperation();  
} catch (error) {  
  console.error(error);  
}
```

অথবা appropriate user-facing handling করো ।

46. `throw "error"` কেন **Avoid** করা ভালো?

Technically JavaScript-এ এটা করা যায়:

```
throw "Something went wrong";
```

কিন্তু preferred:

```
throw new Error("Something went wrong");
```

কারণ `Error` object-এর useful information থাকে:

```
name  
message  
stack
```

47. User-কে Raw Error দেখানো উচিত নয়

Backend বা production application-এ:

```
console.log(error.stack);
```

developer-এর জন্য useful ।

কিন্তু user interface-এ raw stack দেখানো উচিত নয় ।

Instead:

```
Something went wrong. Please try again.
```

অথবা appropriate message ।

48. Sensitive Information

Error message-এর মধ্যে sensitive information রাখা উচিত নয় ।

Avoid:

```
Database password = xyz123
```

বা:

```
JWT secret = ...
```

Error handling-এর সময় security মাথায় রাখতে হবে।

49. Error Handling Best Practices

1. Meaningful error message দাঁও

```
throw new Error("Email is required");
```

2. Appropriate error type ব্যবহার করো

```
throw new TypeError("Expected a number");
```

3. শুধু expected/risky operations handle করো

4. Empty `catch` avoid করো

5. Error context preserve করো

6. User এবং developer-এর error message আলাদা রাখো

7. Production-এ sensitive stack/details expose করো না

50. `try/catch` দিয়ে Logic Error Fix হয় না

এটা খুব important।

```
try {  
  const total = price + quantity;  
} catch (error) {  
  // ...  
}
```

যদি logic ভুল হয় কিন্তু JavaScript error না দেয়, `catch` সেটা automatically ধরবে না।

Example:

```
const price = 100;  
const quantity = 5;  
  
const total = price + quantity;
```

এটা valid JavaScript ।

কিন্তু business logic অনুযায়ী হয়তো:

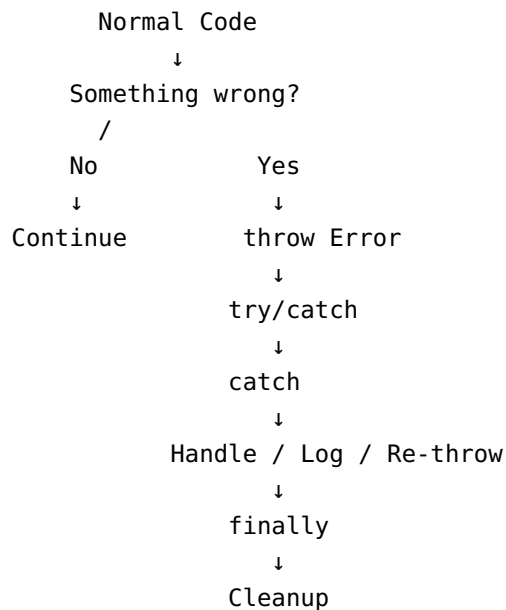
```
price * quantity
```

হওয়া উচিত ছিল।

এটা **logic bug**, runtime exception নয়।

51. Error Handling Mental Model

পুরো process:



52. সবচেয়ে গুরুত্বপূর্ণ Error Types মনে রাখার Trick

TypeError

→ Type ভুল

ReferenceError

→ Reference/variable পাওয়া যায়নি

SyntaxError

→ Syntax ভুল

RangeError

→ Range/value সীমার বাইরে

এক লাইনে:

Type, Reference, Syntax, Range — error-এর কারণের নামেই অনেকটা **clue** থাকে।

53. Practice — Output Prediction

Question 1

```
try {  
  console.log("A");  
  throw new Error("Failed");  
  console.log("B");  
} catch (error) {  
  console.log("C");  
}
```

Output কী হবে?

Question 2

```
try {  
  console.log("A");  
} catch {  
  console.log("B");  
} finally {
```

```
    console.log("C");  
  }
```

Output কী হবে?

Question 3

```
try {  
  throw new Error("Oops");  
} catch (error) {  
  console.log(error.name);  
  console.log(error.message);  
}
```

Output কী হবে?

Question 4

```
try {  
  const data = JSON.parse("{invalid}");  
  console.log(data);  
} catch (error) {  
  console.log(error.name);  
}
```

কোন Error type হবে?

54. Practice — Error Classification

নিচের কোনটা কোন Error?

A

```
console.log(userName);
```

যদি `userName` declare করা না থাকে।

B

```
const user = null;  
  
console.log(user.name);
```

C

```
const = 10;
```

D

```
new Array(-1);
```

নিজে classify করো:

```
TypeError  
ReferenceError  
SyntaxError  
RangeError
```

55. Coding Challenge — Safe Division

এই function তৈরি করো:

```
function divide(a, b) {  
  // your code  
}
```

Requirements:

- `b === 0` হলে error throw করবে
- `a` এবং `b` number কিনা check করবে
- invalid type হলে `TypeError`
- zero division হলে `Error`
- valid হলে result return করবে

Example:

```
divide(10, 2);
```

Expected:

```
5
```

56. Coding Challenge — User Validation

Function:

```
function createUser(name, age) {  
  // your code  
}
```

Requirements:

- `name` empty হলে error
- `age` number না হলে `TypeError`
- age 18-এর নিচে হলে error
- valid হলে object return করবে

Expected:

```
createUser("Sharlar", 31);
```

```
{  
  name: "Sharlar",  
  age: 31  
}
```

57. Coding Challenge — JSON Parser

Function:

```
function parseUser(json) {  
  // your code  
}
```

Requirements:

- JSON parse করবে
- invalid JSON হলে error handle করবে
- user-friendly message return করবে

Example:

```
parseUser( '{"name": "Shariar"}' );
```

58. Interview Questions

Q1. What is error handling?

Answer:

Error handling হলো program-এর unexpected বা exceptional situation detect, handle এবং communicate করার process।

Q2. What is the purpose of `try/catch`?

Answer:

`try` risky code execute করার জন্য এবং `catch` thrown error handle করার জন্য ব্যবহৃত হয়।

Q3. What is `finally`?

Answer:

`finally` এমন code রাখার জন্য ব্যবহৃত হয় যা success বা error—দুই ক্ষেত্রেই execute করা দরকার।

Q4. What does `throw` do?

Answer:

`throw` manually একটি exception/error signal করে এবং normal execution flow interrupt করে।

Q5. Why use `throw new Error()` ?

Answer:

এতে standard Error object পাওয়া যায়, যার `name`, `message`, `stack` ইত্যাদি useful information থাকে।

Q6. Difference between `TypeError` and `ReferenceError`?

Answer:

`TypeError` সাধারণত wrong type-এর operation বোঝায়, আর `ReferenceError` এমন identifier access করার সময় হয় যার binding পাওয়া যায় না।

Q7. Difference between `SyntaxError` and `Runtime Error`?

Answer:

`SyntaxError` code parse করার সময় invalid syntax নির্দেশ করে। Runtime error valid code execute করার সময় ঘটতে পারে।

Q8. Does `finally` always execute?

সাধারণ execution flow-এ `finally` success/error দুই ক্ষেত্রেই execute হয়। তবে process-level termination বা কিছু abrupt execution termination-এর মতো exceptional পরিস্থিতিতে স্বাভাবিকভাবে execute নাও হতে পারে।

59. Quick Revision Table

Concept	মনে রাখবে
<code>try</code>	Risky code
<code>catch</code>	Error handle
<code>finally</code>	Cleanup/final action
<code>throw</code>	Manually error signal
<code>new Error()</code>	Standard error object

Concept	মনে রাখবে
<code>TypeError</code>	Wrong type operation
<code>ReferenceError</code>	Missing/unavailable identifier
<code>SyntaxError</code>	Invalid syntax
<code>RangeError</code>	Invalid range/value
<code>error.name</code>	Error type
<code>error.message</code>	Error description
<code>error.stack</code>	Debugging information
<code>instanceof</code>	Error type check

60. Golden Rules

Rule 1: Error আর bug এক জিনিস নয়।

Rule 2: `try` risky code-এর জন্য।

Rule 3: `catch` error handle করার জন্য।

Rule 4: `finally` cleanup/final action-এর জন্য।

Rule 5: নিজের validation failure signal করতে `throw` ব্যবহার করতে পারো।

Rule 6: সাধারণত `throw new Error("message")` prefer করো।

Rule 7: `TypeError` → type problem।

Rule 8: `ReferenceError` → identifier/reference problem।

Rule 9: `SyntaxError` → syntax problem।

Rule 10: `RangeError` → value/range problem।

Rule 11: `error.message` user/developer-friendly context দিতে useful।

Rule 12: `error.stack` debugging-এর জন্য useful, কিন্তু rawভাবে user-কে expose করা উচিত নয়।

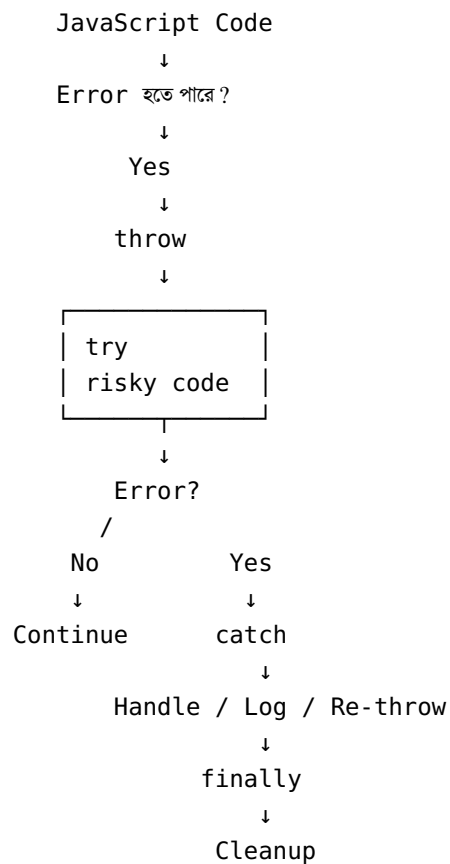
Rule 13: সব code অযথা `try/catch` দিয়ে wrap করো না।

Rule 14: Empty `catch` দিয়ে error silently swallow করো না।

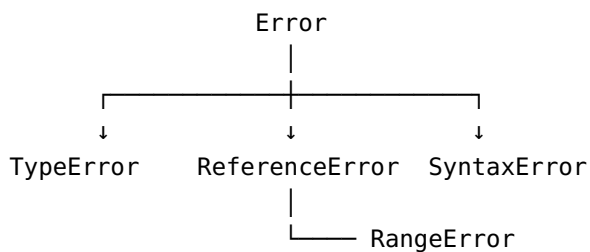
Rule 15: Logic bug automatically `try/catch` দিয়ে ধরা যায় না।

61. Final Mental Model

এই Section-এর পুরো concept একসাথে:



Error types:



আর সবচেয়ে সহজ mental shortcut:

TYPE

→ ভুল type

REFERENCE

→ variable/reference পাওয়া যাচ্ছে না

SYNTAX

→ code-এর grammar ভুল

RANGE

→ value allowed range-এর বাইরে

62. Section 12 Complete Checklist

এই section শেষ করার পর তোমার এগুলো explain করতে পারা উচিত:

- ☐ Error কী
- ☐ Error vs Bug
- ☐ Syntax Error
- ☐ Runtime Error
- ☐ try
- ☐ catch
- ☐ finally
- ☐ Error object
- ☐ error.name
- ☐ error.message
- ☐ error.stack
- ☐ throw
- ☐ throw new Error()
- ☐ throw vs return
- ☐ Error propagation
- ☐ Re-throwing
- ☐ TypeError
- ☐ ReferenceError
- ☐ SyntaxError
- ☐ RangeError
- ☐ URIError
- ☐ AggregateError
- ☐ instanceof
- ☐ JSON parsing error handling
- ☐ API error handling basics

- [] Loading + `finally` pattern
- [] Custom validation errors
- [] Empty `catch` কেন avoid করা উচিত
- [] Error message/security best practices
- [] Logic bug vs runtime error

Section 12 Summary

এই section-এর সবচেয়ে important flow:

```
try
↓
risky operation
↓
error হলে
↓
throw / exception
↓
catch
↓
handle / log / re-throw
↓
finally
↓
cleanup
```

আর error identify করার shortcut:

```
TypeError      → Type problem
ReferenceError → Reference problem
SyntaxError    → Syntax problem
RangeError     → Range problem
```

Section 12 — Error Handling Basics সম্পূর্ণ। 